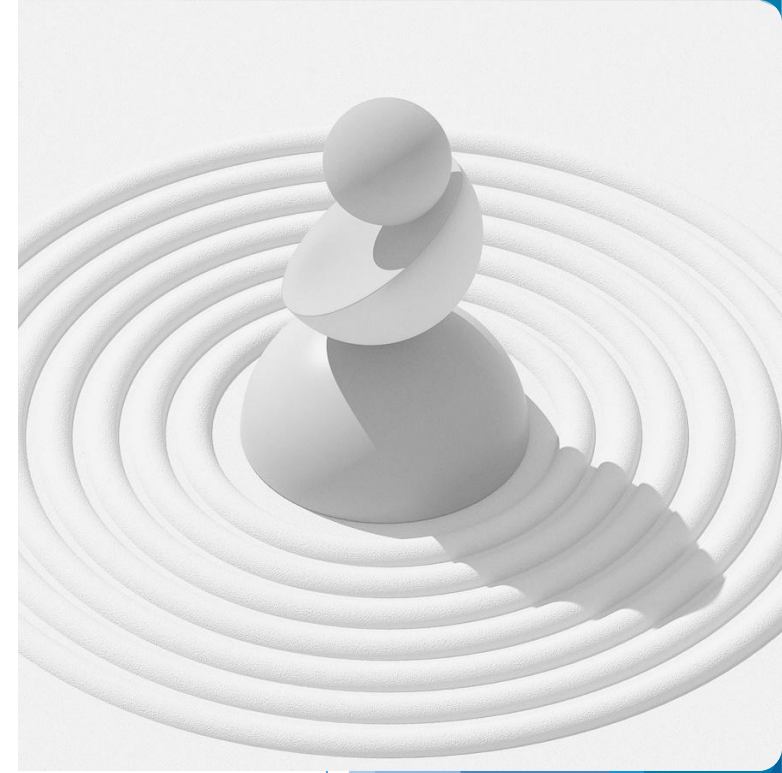


Presented by Nirvana Popescu

# Lesson 1: Introduction to software engineering

# Outline

- ▶ Software engineering as an engineering discipline
- ▶ Importance of structured development methods
- ▶ Real-world relevance



# What Is Software Engineering?

- ▶ According to IEEE, software engineering is defined as:

*“The application of a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software.”*

## Systematic Approach

A systematic approach means that software is built following an organized sequence of steps.

These steps typically include:

- ▶ requirements analysis,
- ▶ architectural and detailed design,
- ▶ implementation,
- ▶ verification and validation,
- ▶ deployment and maintenance.

## Disciplined Development

Being disciplined means adhering to:

- ▶ coding standards,
- ▶ modeling guidelines,
- ▶ safety and certification standards,
- ▶ review and approval processes.

## Quantifiable Results

A quantifiable approach allows engineers to answer questions such as:

- ▶ How much of the system has been tested?
- ▶ How many requirements are fully verified?
- ▶ What is the execution time or worst-case response?

Without measurable data, it is impossible to objectively claim that software is safe, reliable, or ready for certification.

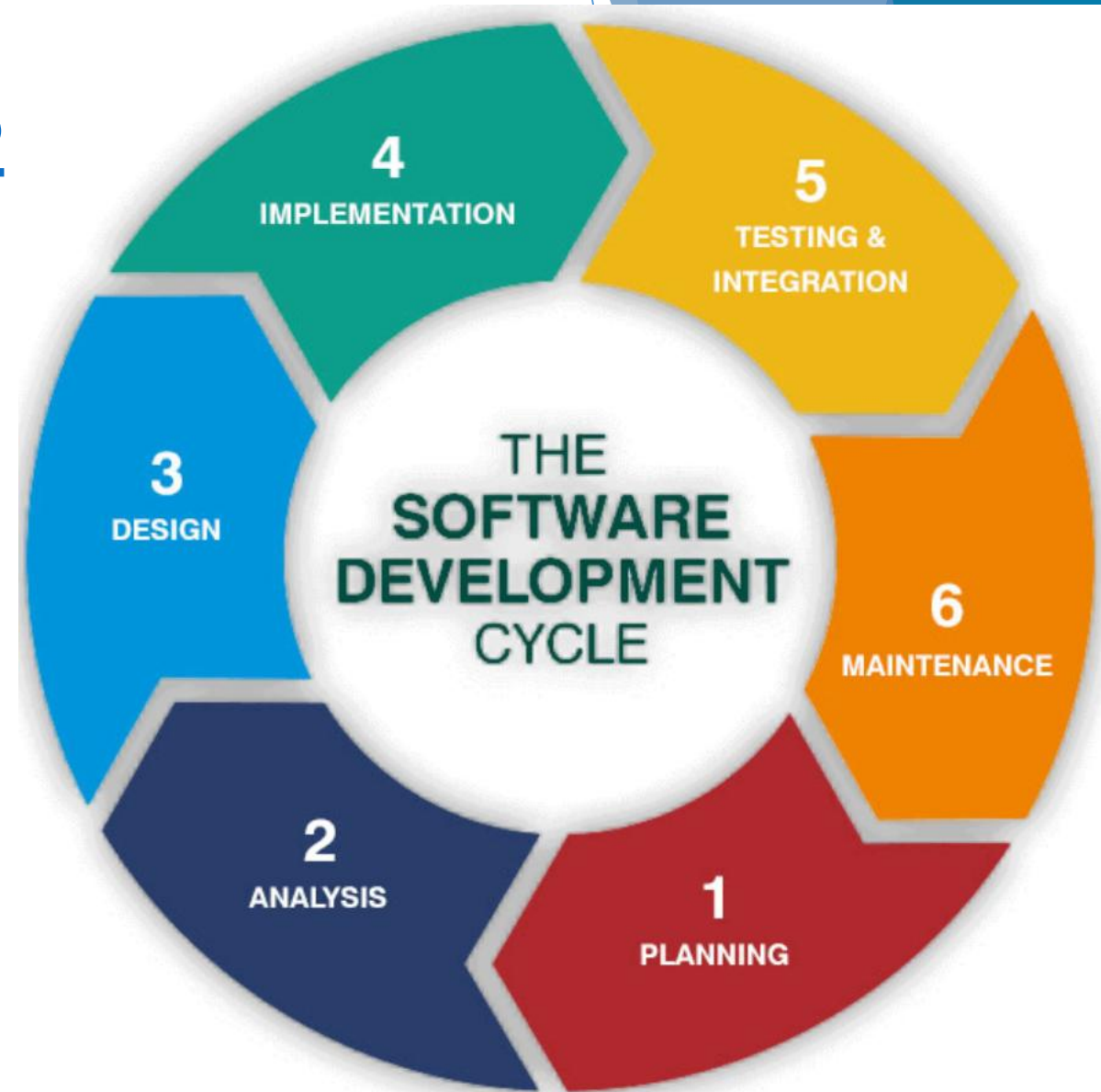


# Software Engineering Covers the Entire Lifecycle

One of the most important ideas in software engineering is that it spans the entire software lifecycle.

This includes:

- ▶ capturing and managing requirements,
- ▶ designing system behavior,
- ▶ implementing logic,
- ▶ verifying correctness,
- ▶ maintaining the system after deployment.



“A complex system that works is invariably found to have evolved from a simple system that worked”

John Gall

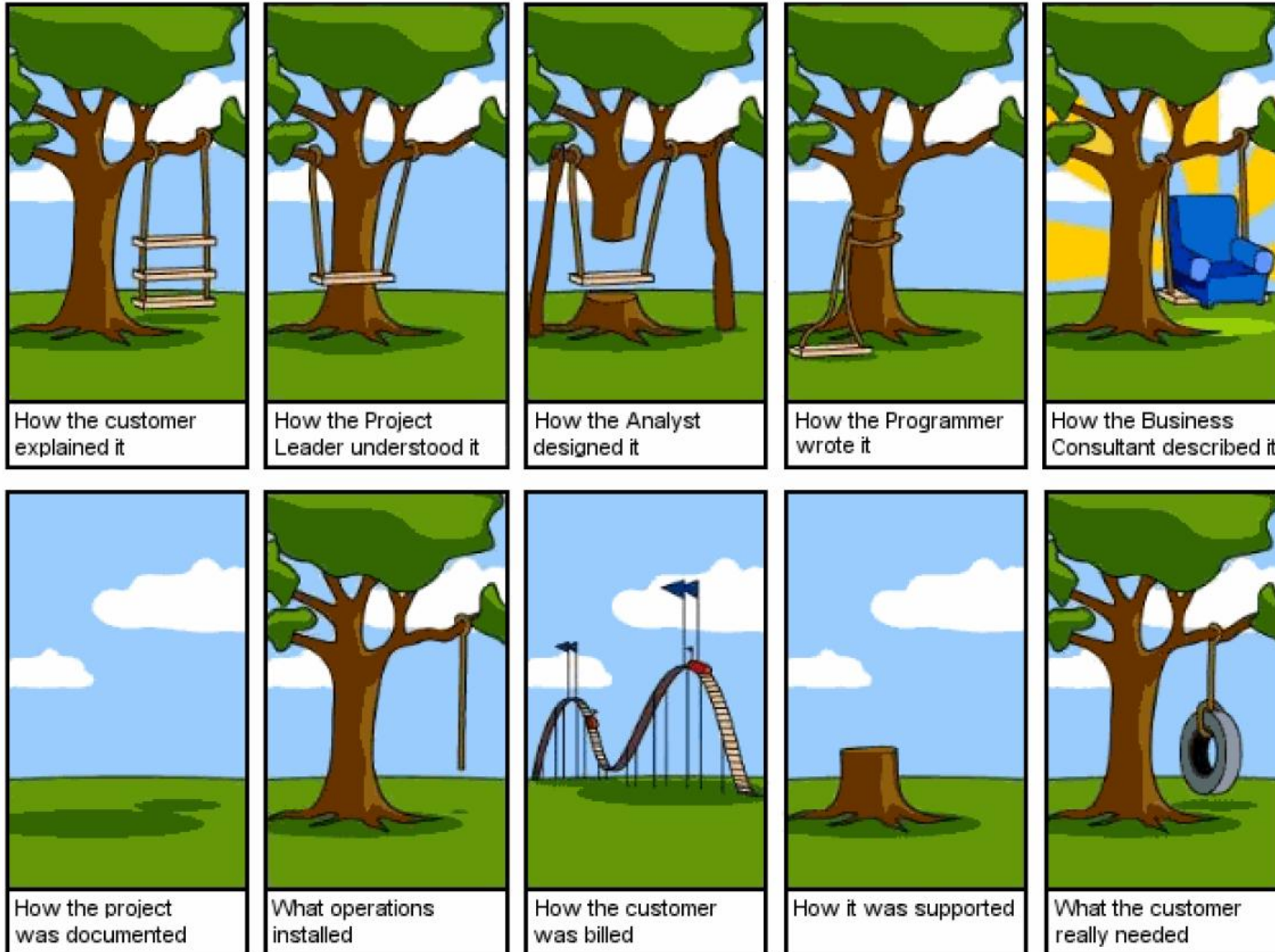
But ... in the history of IT projects, there are many failures.



30-40% of system projects fail before completion.

*Where should the problem be?*

The secret does not reside only in programming, but also in the way the project is organized and developed and the communication between the members of the team.



# Software Engineering vs. Programming

- ▶ Programming focuses on *how* to implement a solution.
- ▶ Software engineering focuses on *whether the solution is correct, safe, maintainable, and verifiable*.



# Characteristics of High-Quality Software

High-quality software is not accidental; it is engineered.

Key characteristics include:

- ▶ **Reliability** - consistent correct behavior over time
- ▶ **Determinism** - predictable outputs for given inputs
- ▶ **Safety** - avoidance of hazardous behavior
- ▶ **Verifiability** - ability to demonstrate correctness with evidence
- ▶ **Traceability** - links between requirements, models, and implementation
- ▶ **Maintainability** - ease of updates without introducing errors



# Why Software Engineering Is Critical in Safety-Critical Systems

Safety-critical systems are systems where a failure can result in:

- ▶ injury or loss of life,
- ▶ major financial loss,
- ▶ environmental damage.

**examples**

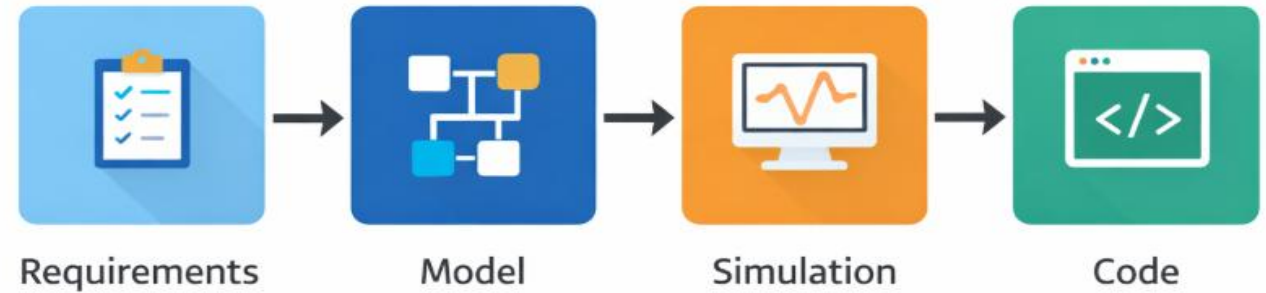


- ☐ flight control systems
- ☐ braking systems
- ☐ railway signaling.

Such systems require:

- ▶ predictability - no unexpected behavior,
- ▶ traceability - every function must be justified,
- ▶ verification evidence - proof that the system meets its requirements.

# What Is Model-Based Design?



- ▶ Model-Based Design (MBD) is a development methodology where system behavior is first described using executable models instead of manually written code.
- ▶ Key ideas:
  - **Graphical system modeling** - engineers describe system logic and behavior using formal models
  - **Early simulation and validation** - system behavior can be tested before implementation
  - **Automatic code generation** - production code can be generated directly from verified models
  - **Improved traceability and reliability** - requirements, models, and code remain connected throughout the development process
- ▶ This approach helps detect errors earlier, reduces development risks, and improves the overall quality of safety-critical software.

# Introduction to ANSYS Scade One

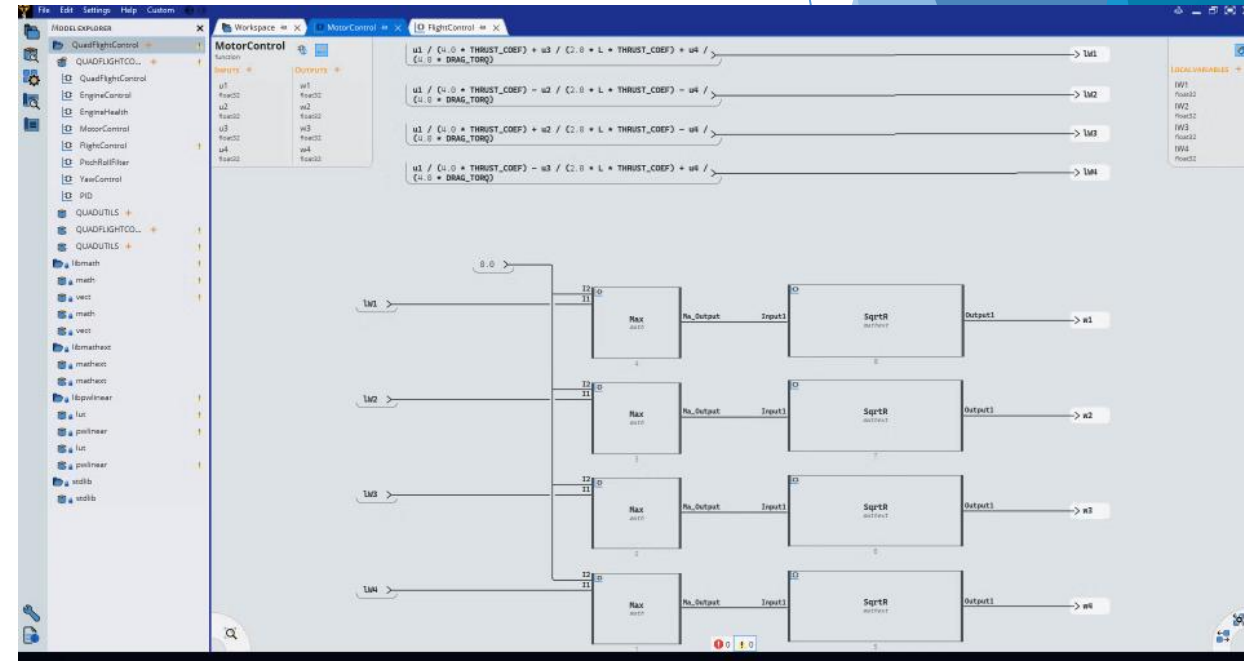
ANSYS Scade One is a model-based development environment designed specifically for safety-critical software.

Instead of writing large amounts of manual code, engineers describe system behavior using formal, executable models.

These models are:

- ▶ deterministic,
- ▶ analyzable,
- ▶ traceable to requirements.

For more info, click [here](#).



# How Scade One Supports Software Engineering Principles

Scade One reinforces software engineering best practices by providing:

- ▶ **Deterministic models.** The behavior of the system is precisely defined and predictable.
- ▶ **Automatic certified code generation.** Code is generated automatically from models, reducing human error and supporting certification.
- ▶ **Strong traceability.** Requirements can be linked directly to model elements and generated code.
- ▶ **Early verification and validation.** Errors can be detected at the model level, long before deployment.

# Where Is ANSYS Scade One Used?

Scade One is widely adopted in industries where software correctness is mandatory:

- ▶ Aerospace - flight control and avionics
- ▶ Automotive - ADAS and electronic control units
- ▶ Railway - signaling and train control
- ▶ Industrial systems - automation and safety controllers



# Summary

- ▶ Software engineering is a disciplined, structured approach to building software that must be correct, safe, and maintainable. In safety-critical systems, these principles are non-negotiable.
- ▶ ANSYS Scade One provides a practical way to apply software engineering principles through model-based design, automatic code generation, and strong verification support.

